

# THE LEGACY OF ALBERTA

## DE OFFICIËLE HINTGIDS

Deel 1 – milde duwtjes, geen antwoorden

---

een gids in twee delen  
voor de zolder van grootmoeder Alberta

## WELKOM, DOORZETTER!

---

Zit je vast op Alberta's zolder? Geen paniek. Dit is precies waarvoor deze gids bestaat. Hier krijg je duwtjes – kleine, milde duwtjes – die je weer op weg helpen zonder dat ze het antwoord verklappen. Zo hoort een echte gids te werken: jij lost het op, wij houden alleen de zaklamp vast.

**Zo gebruik je deel 1.** Zoek het level waar je vastzit. Bij elke puzzel staan twee hints, van vaag naar iets minder vaag. Lees er ééntje. Ga terug naar het spel. Werkt het? Prachtig, sluit de gids. Werkt het nog niet? Lees de volgende hint. Blijf niet doorbladeren tot je per ongeluk te veel weet – dat bederft de pret.

**Onthoud:** in het spel zelf vraag je exact deze hints op, in oplopende stappen. In de terminal en op de zolder typ je daarvoor `?`; in de editor druk je **F1** – daar zet een vraagteken gewoon een vraagteken in de code. Deze gids herhaalt de eerste twee stappen op papier, zodat je ze rustig naast je toetsenbord kan leggen. De derde, meest verklappende stap laten we hier bewust weg – die zoek je in het spel, of, als het écht moet, in deel 2. En deel 2? Dat zit achter een zegel. Om een reden.

**Nog iets:** Alberta houdt bij hoeveel hints je vraagt. Niet om je te straffen – hints kosten je niets en blokkeren niets – maar om je op het einde een warme, droge terugblik te geven. Wie zonder hints geraakt, krijgt de knipoog van de meesterhand. Wie er een hoop nodig had, hoort: “We hebben dit samen gedaan.” Allebei goed. Het spel draait, dat is wat telt.

## DE ZOLDER: HOE JE RONDLOOPT EN FRAGMENTEN VINDT

---

Alberta's zolder is je thuisbasis. Je loopt er rond met de pijltjestoetsen – op een tablet of telefoon met het D-pad onderaan het beeld. **Alle vier de richtingen doe je te voet.** Naar het oosten en het westen loop je gewoon het beeld uit; naar het noorden en het zuiden loop je door de trap: achterin de doorgang gaat ze naar boven, en boven op de overloop breng je het trapgat terug naar beneden. Een geschilderde muur laat je niet door en zegt er niets over – dat is geen bug, dat is een muur.

Typen kan altijd, zoals in een klassiek adventure: `kijk`, `ga oost`, `onderzoek notitieboek`, `open doos`, `ga zitten`, `inventaris`. Verder kan je `herbegin` typen (die vraagt eerst of je het echt meent), `geluid aan / uit` en `crt aan / uit` voor de beeldbuislijnen; **F3** haalt je vorige commando terug in de balk. Typ `help` voor de volledige lijst, en `?` als je niet weet waar het volgende stukje zit.

De grote lijn is simpel en gaat altijd in dezelfde vijf stappen:

1. **Vind het fragment.** Elk level begint met een blad uit Alberta's beschadigde notitieboek. Het eerste ligt gewoon open in de westhoek. De latere bladen zitten in dichtgeplakte, gemerkte dozen, dieper in de zolder (de doorgang in het midden en de overloop boven aan de trap). Eén blad tegelijk: een doos geeft het volgende pas als het hoofdstuk dat je openhebt hersteld is. Vooruitlopen kan dus niet, en dat is met opzet – haar schema loopt op volgorde.

2. **Lees de spread.** Het notitieboek klapt open: Alberta's schets, haar spec, en rechtsonder de regel "Week X in mijn schema" – haar eigen planning voor dat hoofdstuk, en tegelijk een prima maat voor of je er al aan toe bent. Verwacht van die bladen geen uitleg: links staat wat het stuk moet zijn (de klasse, haar velden met hun types, de koppen), rechts wat er stuk of onaf is. Waaróm iets zo werkt, staat in je cursus – en in nood in de hints hieronder. Ze schreef die bladen voor wie ze niet kende: hoofdstuk 1 opent met een aanhef, en daarna zegt ze per hoofdstuk gewoon wat er nog ontbreekt. Met de spatiebalk blader je door; onderaan het linkerblad staat waar die je brengt, en op de laatste bladzijde staat er "spatie: terug". Het boek gaat dan gewoon dicht en je staat nog precies waar je het blad vond.
3. **Ga aan de pc zitten.** Loop zelf naar de werkhoeek in het oosten en typ `ga zitten`. Daar werk je aan haar Java-code. Kwijt? Typ `?` – zolang er een hoofdstuk open staat, wijst de hint je naar de pc.
4. **Los de puzzels op, van boven naar onder.** Meestal één editor-puzzel en twee terminal-puzzels per level. Het menu geeft ze één voor één vrij: wat nog niet aan de beurt is, staat gedoofd met "wacht" ernaast en reageert niet op je klik of cijfertoets. Een taak die je al af hebt, mag je wél opnieuw openen. Alle drie af? Dan is dat hoofdstuk van Alberta's spel hersteld.
5. **Keer terug.** De pc sluit (`Esc`), je voortgang wordt bewaard, en je staat weer op de zolder, klaar voor het volgende fragment. Ga je daarna nog eens zitten terwijl dat hoofdstuk al hersteld is, dan blijft de pc dicht en zegt hij zelf waar het volgende blad ligt.

**Vastgelopen op de zolder zelf?** Bijna altijd is het antwoord: je hebt het volgende blad nog niet, je zit in de verkeerde kamer, of er staat nog een hoofdstuk open. Typ `?`. De hint kijkt naar hoe ver je staat, niet naar waar je staat: ligt er nog een hoofdstuk open op de pc, dan stuurt hij je daarheen; anders zegt hij in welke kamer het eerstvolgende blad ligt – of dat het in déze kamer in een doos zit. Weigert een doos gewoon open te gaan terwijl je `open doos` typt? Dan is het dat derde geval: er is nog een hoofdstuk te herstellen, en de doos zegt zelf welk. Ga eerst dát afwerken aan de pc; daarna geeft ze het blad zonder morren. Die hints zijn gratis en tellen niet mee in Alberta's terugblik. De grote doos in het midden met "BRONCODE" erop telt trouwens pas op het einde; laat die nog even dicht.

Meer dan dit zeggen we niet over de zolder. Het is een kleine, warme ruimte – je hebt ze zo in de vingers.

## LEVEL 1 – KLASSE EN INSTANTIE: ZEVEN UIT ÉÉN VORM

---

*Scharnier: klasse en instantie, velden, constructor, `this`. Speelbaar na week 1.*

Het startpunt, en meteen het fundament van álles. Je herstelt Alberta's `Voorwerp` en schrijft haar `Geitje` uit wat er van de notities rest. Het kernbeeld: een klasse is een **blauwdruk** (het plan dat je één keer tekent); een object is de **doos** die je ernaar bouwt. De constructor vult de velden van zo'n verse doos.

**Puzzel 1 – herstel de constructor van `Voorwerp`.** De waterschade vrát een verwijzing naar de doos zelf op.

- *Hint 1:* Denk aan de blauwdruk en de doos. De constructor vult de velden van een verse doos. Wat gaat waarheen?
- *Hint 2:* Kijk naar de toewijzingen in de constructor. Bij één veld ontbreekt de verwijzing naar de doos zelf, of ze staat net omgekeerd.

**Puzzel 2 – schrijf `Geitje` van nul.** Alberta somt in de kantlijn op wat een geitje heeft – en op bladzijde 2 van het spread staat haar klassekaart met dezelfde drie velden erop; jij tikt de klasse uit.

- *Hint 1:* Een klasse is een blauwdruk: eerst de velden (wat een geitje heeft), dan de constructor die de verse doos vult, dan de getters.
- *Hint 2:* Je mist nog een privaat veld, een toewijzing in de constructor, of een getter. Vergelijk met wat de notitie opsomt (en let op: de gered-vlag begint gewoon op false).

**Puzzel 3 – verklaar in één zin: klasse versus instantie.** Dit beoordeel je daarna zelf tegen Alberta's model.

- *Hint 1:* Blauwdruk en doos: het ene is het plan, het andere het ding dat je ermee bouwt.
- *Hint 2:* Welk woord hoort bij het plan (je tekent het één keer), en welk bij het ding (je maakt er vele van)?

## LEVEL 2 – SIGNATUREN: WAT ERIN GAAT, WAT ERUIT KOMT

---

*Scharnier: signaturen (return vs. `void`), en het verschil tussen attribuut, parameter en lokale variabele. Speelbaar na week 2.*

Nu draait alles om methode-koppen. Het beeld: een methode is een machine. Trechters erin (de parameters), een goot eruit (het returntype) – of niets eruit (`void`). En drie soorten dozen om waarden in te bewaren: attribuut, parameter, lokale variabele.

**Puzzel 1 – Parsons: orden de zoekmethode.** Sleep de stroken in de juiste volgorde. Let op: één strook hoort er níét bij.

- *Hint 1:* De methode begint met haar kop, werkt van boven naar onder, en sluit met een accolade.
- *Hint 2:* Eén strook geeft de verkeerde doos terug – de parameter in plaats van het gevonden object. Die afleider laat je liggen.

**Puzzel 2 – herstel de signaturen van `Speler`.** De koppen zijn tot pap doorgelopen: eentje geeft iets terug maar zegt `void`, of een trechter is verdwenen.

- *Hint 1:* Trechters erin, goot eruit – of niets eruit. Kijk per methode: wat gaat erin, wat komt eruit?
- *Hint 2:* Kijk naar de koppen. Eén geeft iets terug maar zegt `void`; bij een andere is de trechter (de parameter) verdwenen.

**Puzzel 3 – trace: een parameter schaduw een attribuut.** Voorspel de twee getallen die verschijnen.

- *Hint 1:* Drie dozen: attribuut, parameter, lokale variabele. Welke doos bedoelt de naam hier?
- *Hint 2:* Zonder `this` pakt de code de dichtstbijzijnde doos: de parameter. Met `this` de doos van het object (het attribuut).

## LEVEL 3 – VOORWAARDEN: DE DEUR OP SLOT

---

*Scharnier: voorwaarden – validatie, cascade, && / || / !. Speelbaar na week 2.*

De kern van toets 1. Het beeld: een knikkerbaan. De validatie klemt de knikker tussen twee randen; de cascade splitst de baan; && / || / ! sturen welke kant hij op rolt.

**Puzzel 1 – herstel de klem in `setLevenspunten`.** De waarde moet tussen twee randen blijven: nooit onder 0, nooit boven het maximum.

- *Hint 1:* De knikkerbaan: de waarde moet tussen twee randen blijven, nooit onder de ene, nooit boven de andere.
- *Hint 2:* Kijk naar de twee if-controles. Eén rand staat de verkeerde kant op, of een rand ontbreekt helemaal.

**Puzzel 2 – vind de fout: && versus ||.** Een poort mag alleen open als de wolf verslagen is ÉN je de sleutel hebt. Toch klopt er iets niet.

- *Hint 1:* De baan stuurt met && (en) / || (of): welke van de twee laat de knikker door als beide sporen moeten kloppen?
- *Hint 2:* Lees de opgave: de poort mag maar open als alle voorwaarden waar zijn. Kijk naar de operator in de conditie.

**Puzzel 3 – trace: de validatie-cascade op een randwaarde.** Welk woord verschijnt? Let op de randen.

- *Hint 1:* De cascade splitst de baan: de knikker rolt in de eerste tak die klopt, en dan niet meer verder.
- *Hint 2:* Let op de randen: `<= 0` pakt ook net 0, en `< 10` pakt 10 net niet. Waar valt jouw waarde? Loop de takken van boven naar onder.

## LEVEL 4 – REFERENTIES: TWEE PIJLEN, ÉÉN DOOS

---

*Scharnier: referenties (aliasing) en null. Speelbaar na week 3.*

Dit scharnier draagt alle latere lessen. Het beeld: twee variabelen kunnen naar dezelfde doos wijzen – twee pijlen, één doos. Verander je de doos via de ene pijl, dan ziet de andere het ook. En `null` is een pijl die naar geen enkele doos wijst.

**Puzzel 1 – herstel de buur-bedrading (`verbindKamers`).** Elke verbinding tussen kamers loopt twee kanten op. Wat de ene kant legt, moet de andere kant terugleggen.

- *Hint 1:* Twee pijlen, één doos: elke verbinding loopt twee kanten op. Leg je de noord-buur, leg dan meteen de zuid-buur terug.
- *Hint 2:* Kijk naar de takken van de cascade. Bij één richting is de setter verkeerd, zodat de terugweg nooit meer wordt gelegd.

**Puzzel 2 – trace: aliasing.** Twee variabelen wijzen naar dezelfde kamer; je zet de buur via de ene en leest hem via de andere.

- *Hint 1:* Twee pijlen, één doos: lees de tweede regel nog eens – hoeveel kamers maakt deze code eigenlijk aan?

- *Hint 2:* Wat je via `tweede` zet, staat ook in `eerste` – het blijft dezelfde doos.

### Puzzel 3 – verklaar in één zin: wat betekent `null` hier?

- *Hint 1:* Twee pijlen, één doos: waar komt die pijl uit als er in die richting geen kamer bestaat?
- *Hint 2:* Denk aan een kamer zonder uitgang in die richting: waar wijst de buur-referentie dan heen?

## LEVEL 5 – LUSPATRONEN: GEITJE VOOR GEITJE

---

*Scharnier: de lus-romp + patroonkeuze (tellen, totaliseren, opbouwen, filteren, het uiterste). Speelbaar na week 4.*

De kern van toets 2. Elke lus volgt een patroonkaart. Kies eerst de kaart, dan schrijf je de romp bijna vanzelf.

**Puzzel 1 – schrijf twee lus-methoden.** `telWapens` telt (een teller die ophoogt); `sterksteVoorwerp` zoekt het uiterste (onthoud de beste tot nog toe).

- *Hint 1:* Welke patroonkaart? `telWapens` is de tel-kaart; `sterksteVoorwerp` is de uiterste-kaart.
- *Hint 2:* Begin `telWapens` met een teller op 0 en hoog op bij een treffer. Begin `sterksteVoorwerp` met `null` en vervang zodra je iets sterkers ziet.

**Puzzel 2 – Parsons: de string-builder.** Bouw een kamer-regel die met elke ronde langer wordt. Eén strook hoort er niet bij.

- *Hint 1:* De opbouw-kaart: je maakt een regel die met elke ronde langer wordt.
- *Hint 2:* Eén strook overschrijft de regel in plaats van eraan toe te voegen – dat is de klassieke opbouw-fout, en die hoort er niet bij.

**Puzzel 3 – welke patroonkaart?** Een lus uit het schade-overzicht. Kies uit tellen, totaliseren, opbouwen of het uiterste.

- *Hint 1:* Kijk wat de lus met elk element doet: telt ze er één bij, of telt ze de waarde zélf op?
- *Hint 2:* Bij `som = som + schadelog[i]` groeit `som` met de waarde, niet met één per element.

## LEVEL 6 – INDEX EN OFF-BY-ONE: DE LAATSTE PLANK

---

*Scharnier: index en off-by-one; welke lus kies ik. Speelbaar na week 5.*

Toets 2 en de eindtoets, allebei. Het beeld: een lijst is een plankenbrug boven een ravijn. De eerste plank is nummer 0; de laatste is `size()` min één. Eén plank te ver en je ligt in het water.

**Puzzel 1 – herstel de verwijder-lus.** In `verwijderVoorwerp` loopt een lus één plank te ver, of het is de verkeerde soort lus.

- *Hint 1:* De plankenbrug: de eerste plank is 0, de laatste is `size()` min één. Eén plank te ver en je ligt in het water.
- *Hint 2:* Kijk naar de lusgrens, of naar de soort lus: klopt `<` versus `<=`, en is het wel de lus die hier past?

**Puzzel 2 – trace: de laatste afgedrukte index.** Voor een lijst met N voorwerpen: welk getal wordt als laatste afgedrukt?

- *Hint 1:* Tel de planken vanaf 0, niet vanaf 1.
- *Hint 2:* De lus stopt zodra `i` niet meer kleiner is dan `size()`. De laatste `i` die nog gedrukt wordt, is `size()` min één.

**Puzzel 3 – vind de fout: de rondelus.** Een gevechtslus hoort precies vijf rondes te spelen (0 tot en met 4). Toch klopt er iets niet.

- *Hint 1:* De plankenbrug: hoe ver loopt de lus, en is dat één plank te ver?
- *Hint 2:* Kijk naar de grens van de `for`-lus op regel 1, niet naar de romp.

## LEVEL 7 – ZOEKEN EN DE DUBBELE PIJL: WAAR HET JONGSTE ZIT

---

*Scharnier: zoeken + de dubbele pijl (een ketting van getters). Speelbaar na week 6.*

Het laatste hoofdstuk, en de kern van de eindtoets. Een zoeklus is een speurtocht: hij geeft het gevonden object terug, of `null` als de tocht doodloopt. En soms volg je twee pijlen na elkaar – een ketting van getters, die je null-veilig moet houden.

**Puzzel 1 – schrijf de zoeklus (`zoekGeitje`).** Loop de lijst af en geef het geitje met de gezochte naam terug, of `null` als het er niet is.

- *Hint 1:* De speurtocht: een zoeklus loopt de lijst af, geeft het gevonden object terug, of `null` als de tocht doodloopt.
- *Hint 2:* Loop met een `for`-lus over de geitjes, vergelijk elke naam, en geef bij een treffer meteen het geitje terug. Kom je aan het einde zonder treffer, dan `null`.

**Puzzel 2 – herstel de null-veilige keten.** `geitje.getSchuilplaats().getKamer()` – maar een geitje zonder schuilplaats heeft geen kamer om naar te wijzen.

- *Hint 1:* De dubbele pijl: soms wijst de eerste pijl naar `null`, en dan is er geen tweede pijl om te volgen.
- *Hint 2:* Er ontbreekt een null-controle vóór de tweede pijl, of de keten mist een schakel.

**Puzzel 3 – trace: de geketende getter, met het null-geval.** Twee geitjes, eentje met en eentje zonder schuilplaats. Wat verschijnt er?

- *Hint 1:* De speurtocht met de dubbele pijl: soms wijst de eerste pijl naar `null`.
- *Hint 2:* Controleer eerst of de schuilplaats `null` is. Is ze `null`, dan stopt de keten op “nog niet gevonden”.

## HET EINDSPEL: SEVEN LITTLE GOATS

---

Level 7 af? Dan boot de pc Alberta's spel als speelbare simulatie in de terminal, en speel je eindelijk *Seven Little Goats* uit. Je bent Roodkapje, intussen de dorpsexpert die je nooit wilde zijn. De wolf van het oude verhaal is weg; deze is zijn neef, jong, en hij heeft het verhaal gelezen als een

handleiding. Hij slokte zes geitjes op; het jongste kroop in de klokkast en riep tot iemand het hoorde. Je volgt zijn spoor van het geitenhuisje tot aan de rivier. Wat je daar met hem doet, bepaal jij.

We houden het hier bewust luchtig – geen kaart, geen recept, geen exacte commando's. Dat is de fijne ontdekking, die willen we je niet afpakken. Wel drie eerlijke richtwijzers:

- ▶ **Verken en verzamel.** Loop de kamers af, pak wat je tegenkomt, hou met inventaris en stats bij wat je draagt, en praat met wie er is. Sommige voorwerpen genezen je, eentje telt mee bij elke aanval, en eentje uit grootmoeders huisje opent meer dan stof. Zonder mandje krijg je de koeken van het dorpsplein niet mee, en een raaf in het bos ruilt graag.
- ▶ **Het wolfgevecht telt.** Vijf rondes, een vast patroon, geen toeval – wie rékent, wint. Zorg dat je scherp bent voor je vecht typt: het juiste gereedschap op zak maakt het verschil tussen net winnen en kopje-onder gaan. De jachthond bij de molen is optioneel; een omweg, geen verplichting.
- ▶ **Sparen of afmaken is een échte keuze.** Als de wolf op het einde smeekt, is wat je dan kiest geen detail – het bepaalt mee welk van de vier eindes je krijgt. Er is een best einde dat twee dingen tegelijk vraagt, een kille variant, een genadige, en natuurlijk een game over. Welke je “hoort” te spelen, beslis jij. Ze zijn geen van alle fout.

Vast in de sim? Typ ? voor een hint die bij je huidige plek past, of opties voor een compleet overzicht van wat hier kan – dat laatste is een cheatcode, en dat weet het spel. Gebruik hem met mate.

En de exacte weg naar elk van de vier eindes, met alle commando's op een rij? Die staat in deel 2. Achter het zegel. Je weet wat je te doen staat – of net niet te doen.

*Veel succes, doorzetter. Alberta kijkt mee.*